PROYECTOS EN TECNOLOGÍA

Métodos ágiles, Scrum y Kanban

Geoffrey Hecht

DESARROLLO EN EQUIPO

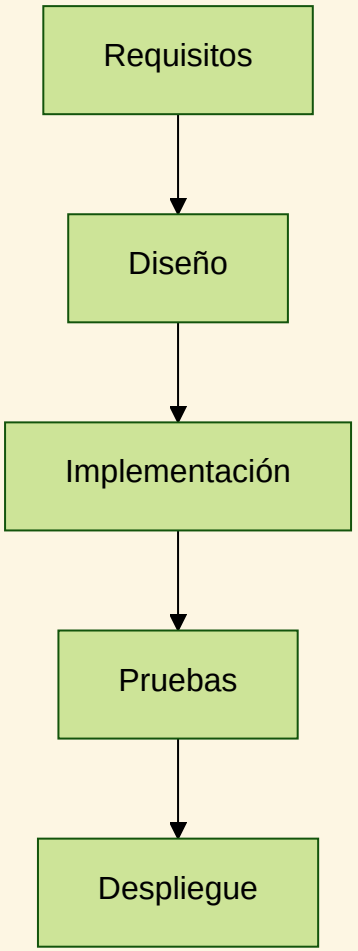
- Llevar a buen puerto un proyecto de tecnología en equipo puede ser difícil, y lo es más a medida que el equipo crece
- ¿Cómo lo han hecho hasta ahora?
- ¿Cuáles fueron sus problemas?
- La dificultad suele estar en la **asignación de tareas** y en saber **quién hace qué**
- En este curso trabajaremos con **Scrum**, un método ágil, en un formato simplificado

MODELOS DE CICLO DE VIDA

Todo proyecto recorre las mismas actividades: entender qué se necesita, diseñar, construir, probar y entregar. Un **modelo de ciclo de vida** define en qué **orden** y con qué **frecuencia** se recorren.

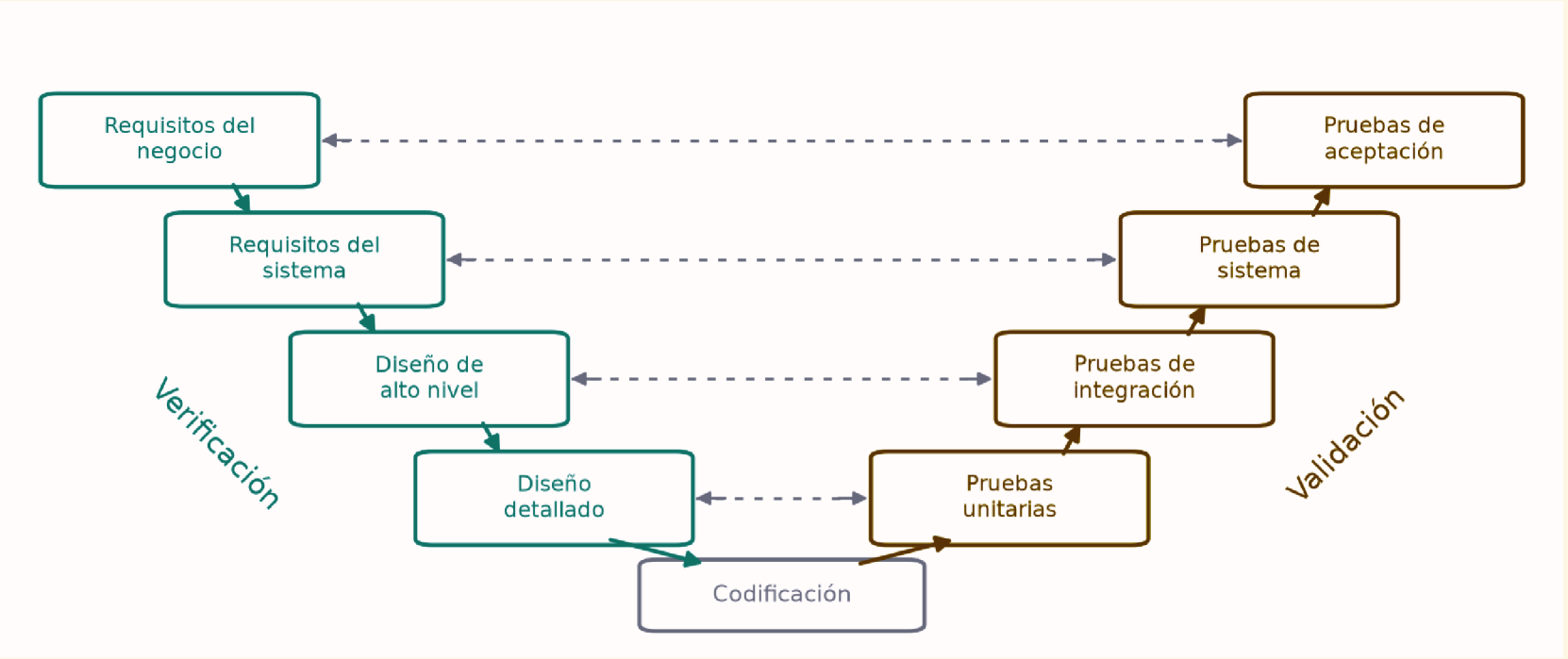
- Existen muchos: cascada, en V, espiral, incremental, iterativo, RAD (*Rapid Application Development*), ágil…
- Aquí veremos tres, en breve, para entender de dónde vienen los métodos ágiles

MODELO EN CASCADA (*WATERFALL*)



- Cada fase **termina** antes de que empiece la siguiente, y produce un documento que la siguiente toma como entrada
- El flujo va en **un solo sentido**: no hay vuelta atrás prevista
- Funciona cuando los requisitos son **estables y bien conocidos** desde el principio
- El resultado funcionando aparece **recién al final**
- En ciencia de datos, la misma secuencia: definición del problema, recopilación, limpieza, exploración, modelado, evaluación, despliegue y monitoreo

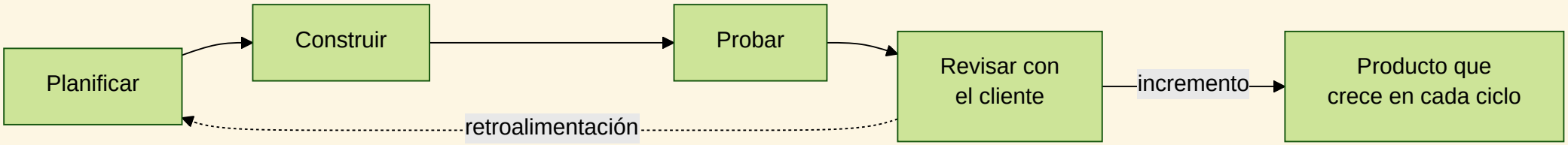
MODELO EN V



- Es la cascada **doblada**: a cada fase de especificación y diseño le corresponde un **nivel de prueba**
- Las pruebas se **diseñan al mismo tiempo** que su nivel: las de aceptación con los requisitos, las unitarias con el diseño detallado
- Sigue siendo **lineal**, con una sola entrega al final

¿Cuáles son los límites de estos modelos lineales?

ITERATIVO E INCREMENTAL



- El proyecto se divide en **ciclos cortos**, cada uno con su diseño, construcción y prueba
- Cada iteración entrega **un resultado que funciona** (software, análisis, modelo), en lugar de un documento
- El cliente ve el producto temprano, y su retroalimentación entra en la iteración siguiente
- Es la base de los **métodos ágiles**

MODELOS FRENTE AL CAMBIO

	Cascada	Modelo en V	Iterativo
Entregas	una, al final	una, al final	una por iteración
Pruebas	fase final	planificadas desde el inicio	en cada iteración
Retroalimentación	al final	al final	continua
Costo del cambio	muy alto	alto	moderado
Requisitos	fijos y conocidos	fijos y conocidos	pueden evolucionar

ORIGEN DE LAS METODOLOGÍAS ÁGILES

- Muchos requisitos de software cambian constantemente
- Los métodos tradicionales tipo cascada son rígidos ante el cambio
- La meta de la cascada es entregar un software completo y sin fallas en una sola entrega
- Supone que una documentación inicial exhaustiva garantiza mayor calidad y un desarrollo más rápido
- Los clientes no siempre saben qué quieren al inicio
- *Feedback* (retroalimentación) tardío implica correcciones costosas

MANIFIESTO ÁGIL

- Surgió en 2001, cuando 17 expertos en desarrollo de software se reunieron para debatir el futuro de la disciplina
- Estaban frustrados por procesos pesados y burocráticos que frenaban la innovación y producían resultados decepcionantes

> "El movimiento Ágil no es anti-metodología, de hecho, muchos de nosotros queremos devolverle la credibilidad a la palabra metodología. Queremos restablecer un equilibrio. Adoptamos el modelado, pero no para archivar un diagrama en algún repositorio corporativo polvoriento. Adoptamos la documentación, pero no cientos de páginas de tomos que nunca se mantienen y que rara vez se usan. Planificamos, pero reconocemos los límites de la planificación en un entorno turbulento."

VALORES DEL MANIFIESTO

Los firmantes valoran más lo de la izquierda, sin descartar lo de la derecha:

Se valora más	Sobre
Individuos e interacciones	Procesos y herramientas
Software funcionando	Documentación exhaustiva
Colaboración con el cliente	Negociación contractual
Respuesta ante el cambio	Seguir un plan

Fuente: agilemanifesto.org, que también enumera 12 principios

MÉTODOS ÁGILES EN RESUMEN

- **Iterativo** (repetir y refinar en ciclos) e **incremental** (construir por partes chicas): entregas frecuentes de valor
- **Colaborativo**: equipos multifuncionales trabajando juntos
- **Adaptativo**: el cambio se incorpora al plan en vez de evitarse
- **Centrado en personas**: potencia la creatividad del equipo

ÁMBITO DE USO

Útil cuando	Poco útil cuando
Los requisitos son inciertos o cambiantes	Los requisitos son fijos y bien definidos desde el inicio
El proyecto es innovador o experimental	El entorno está altamente regulado
El equipo es pequeño y comunicativo	El proyecto y el equipo son grandes (difícil de implementar)

EJEMPLOS DE MÉTODOS ÁGILES

- **Scrum**: marco iterativo que trabaja en ciclos llamados *sprints* para entregar versiones incrementales del producto
- **Kanban**: flujo de trabajo continuo que limita el trabajo en curso para mejorar la productividad
- **XP** (*Extreme Programming*): enfatiza prácticas técnicas como la programación en pareja y las pruebas constantes para adaptarse a los cambios
- **SAFe** (*Scaled Agile Framework*): marco para escalar ágil en grandes organizaciones, con alineamiento estratégico
- …

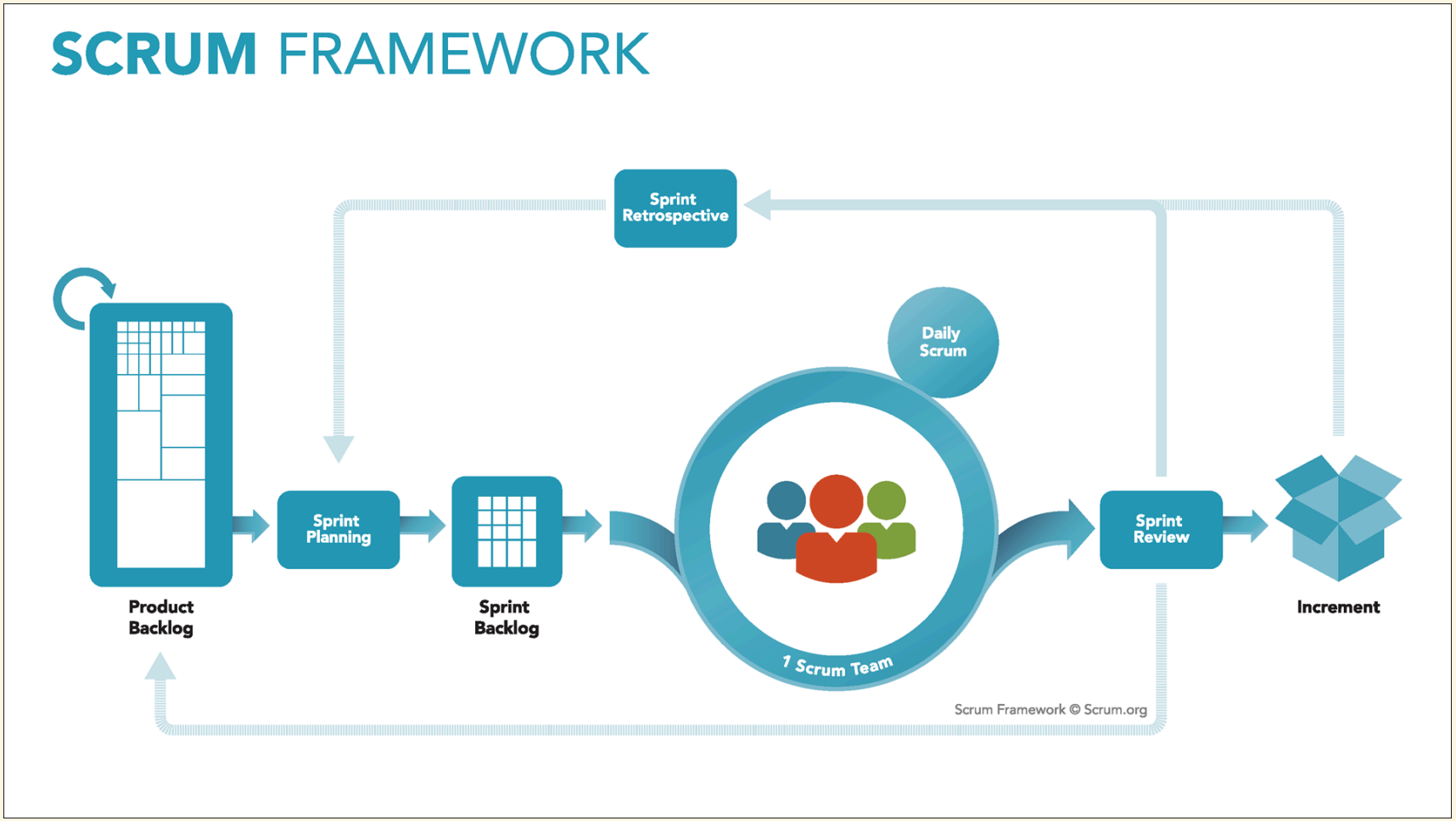
SCRUM

- Marco (*framework*) ágil para la gestión de proyectos, el más popular
- Enfoque empírico: **transparencia**, **inspección**, **adaptación**
- Nació para el desarrollo de software, hoy se usa en múltiples contextos
- Tres roles:
  - **Product Owner** (PO, dueño del producto): define qué construir y prioriza
  - **Scrum Master**: facilita el proceso y elimina impedimentos
  - **Desarrolladores**: construyen el producto
- Objetivo: responder a la **visión del producto**, una declaración concisa sobre el objetivo a largo plazo del producto y el valor que entregará a quienes lo usan

SCRUM Y CIENCIA DE DATOS

- Ayuda a gestionar la complejidad e incertidumbre propias de los proyectos de ciencia de datos
- Facilita la adaptación a los cambios frecuentes en los requisitos o en los datos
- Facilita la planificación iterativa y la priorización constante
- Enfoca al equipo en entregables claros y medibles en periodos cortos
- Alinea las tareas del equipo con los objetivos del negocio, priorizando las actividades de mayor valor
- Se puede combinar con metodologías específicas de ciencia de datos

MARCO SCRUM


SCRUM FRAMEWORK
Sprint Retrospective
Daily Scrum
Product Backlog
Sprint Planning
Sprint Backlog
1 Scrum Team
Sprint Review
Increment
Scrum Framework © Scrum.org

PRODUCT BACKLOG

- Lista ordenada de todo lo que se necesita en el producto
- Única fuente de verdad para los requisitos del producto
- Documento vivo que evoluciona constantemente
- Nunca está “completo”: siempre emerge nueva información
- **Ordenado por valor**: los ítems más valiosos van arriba
- **Estimado**: cada ítem tiene una estimación de esfuerzo (difícil al principio)

HISTORIAS DE USUARIO

Los elementos del backlog suelen ser **historias de usuario** (*user stories*, US).

> ⓘ **Plantilla para software**
>
> “Como [usuario], quiero [funcionalidad] para [beneficio]”

> ⓘ **Plantilla para ciencia de datos**
>
> “Como [stakeholder], quiero [saber / aprender / predecir / analizar algo], para poder [lograr un objetivo de negocio o tomar una decisión]”

Cada historia lleva **criterios de aceptación**: condiciones concretas que indican que está completa, a menudo escritas como escenarios de uso.

EJEMPLOS DE HISTORIAS

Siglas: **SLEP** (Servicio Local de Educación Pública), **MINEDUC** (Ministerio de Educación), **SNED** (Sistema Nacional de Evaluación del Desempeño de los establecimientos).

- **US1**: Como analista del SLEP, quiero identificar establecimientos educacionales rurales con bajo rendimiento SNED y alta vulnerabilidad socioeconómica, para poder priorizar la asignación de subsidios de mejoramiento educativo en mi territorio
- **US2**: Como investigador en políticas educativas del MINEDUC, quiero predecir qué niños en educación preescolar tienen mayor riesgo de bajo rendimiento escolar a partir de su desarrollo socioemocional y sus características familiares, para poder implementar intervenciones tempranas focalizadas antes de que ingresen a educación básica
- **US3**: Como planificador territorial del MINEDUC, quiero mapear brechas de calidad educativa por zona geográfica combinando indicadores SNED, características socioeconómicas y accesibilidad territorial, para poder optimizar la distribución regional de recursos y programas de apoyo

DEFINITION OF DONE

- En un proyecto, a veces es difícil ponerse de acuerdo sobre qué se considera **terminado**
- La **DoD** (*Definition of Done*, definición de terminado) es la lista de requisitos que debe cumplir cualquier entrega para considerarse completa
- Se aplica a todas las historias, a diferencia de los criterios de aceptación, que son propios de cada una

EJEMPLO DE DOD EN SOFTWARE

- El código cumple con los requerimientos funcionales especificados
- Se implementaron todos los criterios de aceptación definidos
- El código sigue las convenciones y estándares de codificación del equipo
- El software es *responsive* y compatible con dispositivos móviles y de escritorio
- Se implementó el manejo de errores
- Pruebas unitarias escritas y aprobadas (cobertura mínima de 80 %)
- Revisión de código (*code review*) completada y aprobada por al menos otro desarrollador
- Documentación de las funciones complejas actualizada
- …

EJEMPLO DE DOD EN CIENCIA DE DATOS

- La hipótesis de negocio está claramente establecida
- Las fuentes de datos relevantes están identificadas, documentadas y validadas
- Los pasos de limpieza de datos son reproducibles
- Los resultados exploratorios y finales están respaldados por código y se pueden reproducir
- Las métricas del modelo o del análisis están documentadas
- Los resultados y las conclusiones se resumen en un formato accesible

Esta DoD viene incluida en la plantilla de historia de usuario de los repositorios del curso.

STORY POINTS

- Unidad de medida **relativa** para estimar el esfuerzo de una historia de usuario
- Se expresan en puntos, sin conversión a horas ni días
- Combinan complejidad, esfuerzo e incertidumbre
- Cada equipo tiene su propia escala
- Se suele usar la secuencia de Fibonacci: 1, 2, 3, 5, 8, 13, 21…
- **Refinamiento continuo**: al terminar el sprint se comparan las estimaciones con lo ocurrido

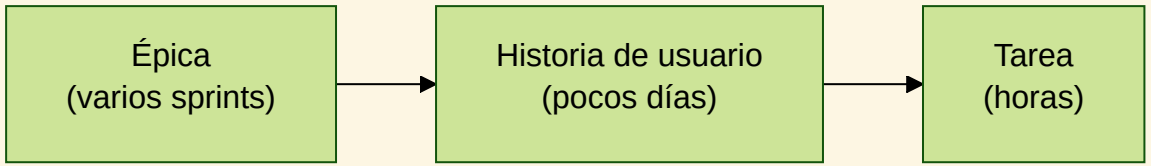
PLANNING POKER

Técnica de estimación en grupo (planningpokeronline.com):

1. Se presenta la historia al equipo
2. Cada miembro elige una carta en secreto
3. Se revelan las estimaciones al mismo tiempo
4. Se discuten las diferencias significativas
5. Se repite hasta alcanzar consenso

Revelar al mismo tiempo evita que la primera opinión expresada ancle a las demás.

TAREAS



- Las tareas describen los pasos que siguen los desarrolladores para completar una historia de usuario
- Generalmente se estiman en horas de trabajo

EJEMPLOS DE TAREAS

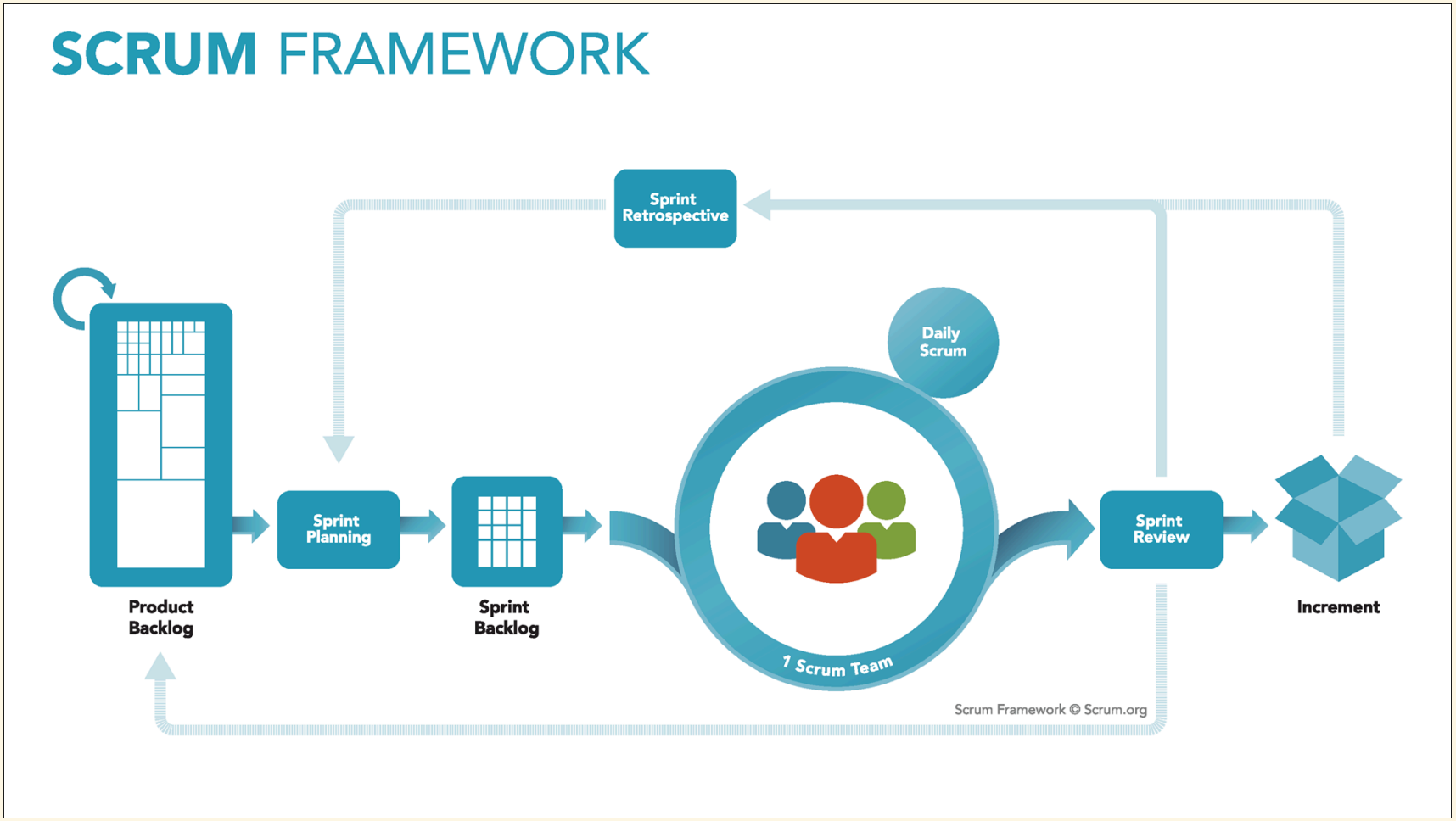
Implementar paginación básica

- Botones Anterior / Siguiente
- Mostrar el número de página actual
- Navegación lineal simple

Analizar la calidad de datos y la integridad referencial entre tres datasets

- Script que verifique que todos los `idrbd` de los datasets `mn` y `sned` existen en el dataset `geo`
- Reporte de cobertura con el % de establecimientos con datos completos en los tres datasets
- Identificación y documentación de valores atípicos en variables clave
- Análisis de valores faltantes por variable

EVENTOS SCRUM



EVENTOS

Sprint Planning (planificación, prepara el backlog)

- ¿Qué software (análisis) terminado entregaremos en este sprint?
- ¿Cómo lo vamos a entregar?

Daily Scrum (reunión diaria, sigue el progreso de las tareas)

- ¿Cuál es nuestro plan para llegar a *Done*?
- ¿Estamos progresando adecuadamente?
- ¿Hay algo que nos impida avanzar?

Sprint Review (revisión, al final del sprint)

- "Este es el software (análisis) terminado y reproducible que desarrollamos"
- "Estos son los *insights* y conclusiones que obtuvimos"
- "Esto es lo que planeamos construir (analizar) a continuación"
- "¿Algún comentario?"

Sprint Retrospective (retrospectiva, al final del sprint)

- ¿Cómo nos fue con la entrega?
- ¿Qué nos causó problemas?
- ¿Qué vamos a mejorar en el próximo sprint?

ROLES

Rol	Responsabilidades
Product Owner	Gestiona el Product Backlog: prioriza, refina y lo mantiene actualizado. Define la visión del producto y comunica el valor de negocio. Acepta o rechaza el trabajo según los criterios de aceptación. Representa al cliente, a los usuarios finales y a los stakeholders
Scrum Master	Facilita el proceso y vela por que el equipo siga las prácticas de Scrum. Acompaña al equipo en la mejora continua y en la resolución de impedimentos. Protege al equipo de distracciones y obstáculos externos
Equipo de desarrollo	Entrega incrementos que cumplen la DoD. Se autoorganiza: decide cómo realizar el trabajo y estima el esfuerzo. Colabora para alcanzar el objetivo del sprint. Identifica mejoras en el proceso y en el producto

TABLERO KANBAN

- Herramienta visual para gestionar y seguir el progreso de un proyecto o proceso
- “Kanban” viene del japonés y significa “tarjeta que se puede ver”; el método nació en Toyota y hoy se usa ampliamente en gestión de proyectos y desarrollo de software
- **Columnas** = etapas del flujo de trabajo, **tarjetas** = tareas
- Cada tarjeta avanza de columna en columna, por ejemplo *To Do* (por hacer), *Doing* (en proceso), *Done* (terminado)
- Cada tarjeta se asigna a un miembro del equipo: se ve en todo momento cómo avanza el proyecto y quién hace qué
- Se usa en el método Kanban y también en otros métodos, entre ellos Scrum

EJEMPLO DE TABLERO

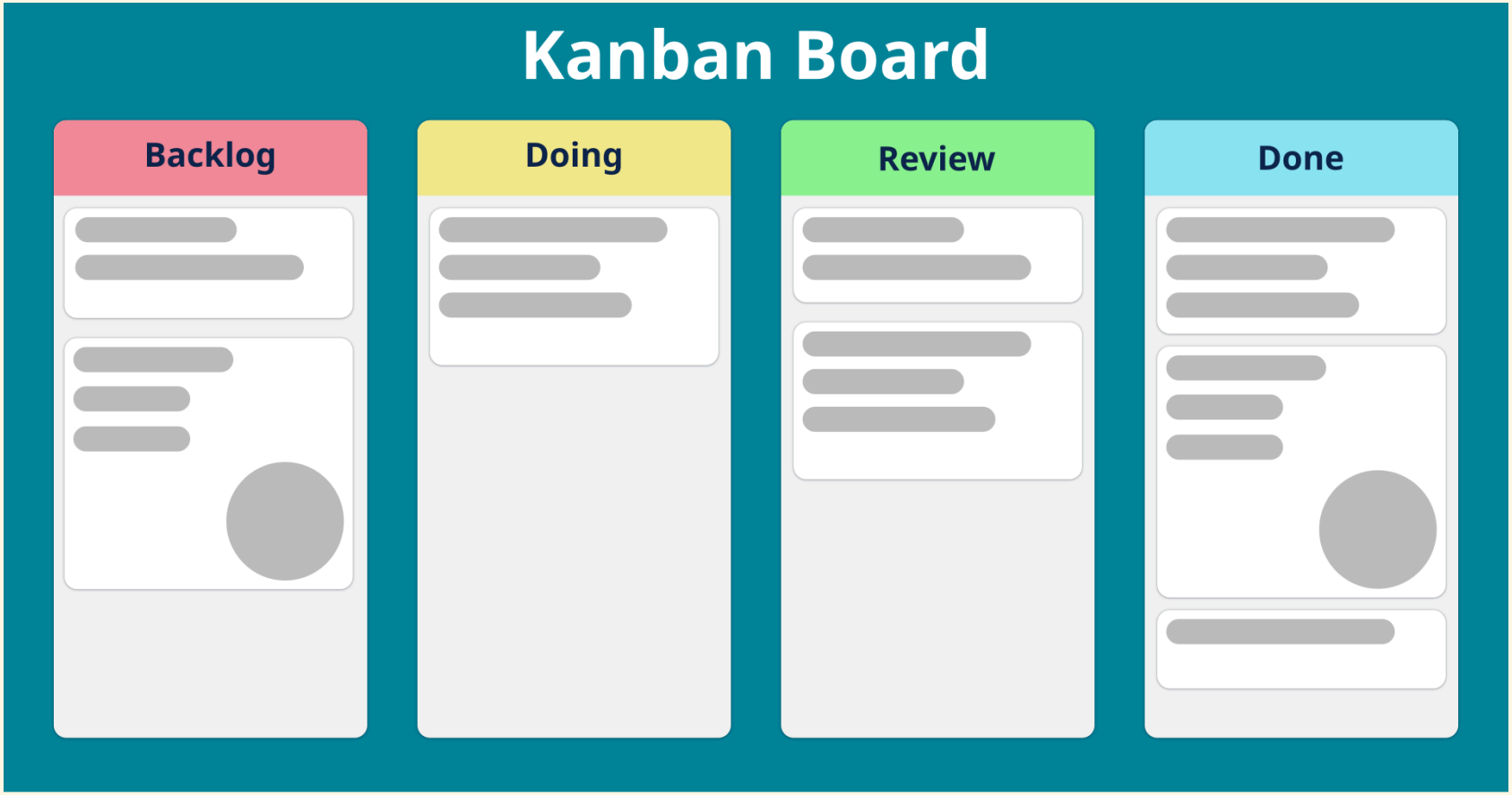


Imagen: Jennifer Falco, CC BY 4.0, Wikimedia Commons

SCRUM EN EL CURSO

- Usaremos **GitHub Projects** (ver demo)
- Trabajaremos en **2 sprints**
- El PO y el Scrum Master también forman parte del equipo de desarrollo
- El backlog es la pieza central del trabajo
- Tablero Kanban con al menos 3 columnas: *To Do*, *Doing*, *Done*
- Se pueden crear subtareas
- El tablero se mantiene actualizado en todo momento, y cada tarea se asigna al pasar a *Doing*
- El tablero se usa en las evaluaciones

ACTIVIDAD

- Revisar los proyectos en GitHub: github.com/orgs/TECH-MACI/projects
- Elegir un Product Owner y un Scrum Master por grupo (cambian en cada sprint)
- Empezar a escribir sus historias de usuario

REFERENCIAS

- Managing your projects with GitHub
- Manifiesto Ágil
- The Scrum Guide
- Benday: GitHub Projects